

IFOSUP WAVRE

PROJET DE DÉVELOPPEMENT

Chat Normal

SGBD

VAN DER VEEN GEORGÉ

Année académique 2025-2026

TABLE DES MATIÈRES

1 Introduction	2
1.1 Technologies utilisées	2
2 Thème et identité	2
2.1 Thème choisi	2
2.2 Personnalisation de l'IA	4
2.3 Personnalité de l'IA & instructions système	5
2.4 Branding	6
3 Modèle de données & architecture	7
3.1 Diagramme de classe	7
3.2 Tables et relations	7
3.3 Contraintes	8
3.4 Documentation du code & gestion des erreurs	8
4 Fonctionnalités implémentées	9
4.1 Fonctionnalités obligatoires	9
4.2 Fonctionnalités supplémentaires	11
5 Difficultés rencontrées	11
6 Tests	12
7 Utilisation de l'IA	12
8 Réflexion, améliorations & conclusion	13
8.1 Glossaire	13

TABLE DES FIGURES

Fig. 1 Interface avec le niveau de folie le plus bas.	3
Fig. 2 Interface avec un niveau de folie bas.	3
Fig. 3 Interface avec un niveau de folie moyen.	4
Fig. 4 Interface avec un niveau de folie maximal.	4
Fig. 5 Aperçu de la modale de configuration.	5
Fig. 6 Diagramme de classe.	7
Fig. 7 Gestion des erreurs.	9
Fig. 8 Selecteur de model.	10
Fig. 9 Résultat des tests.	12

1 INTRODUCTION

Ce projet, réalisé dans le cadre du cours de développement et SGBD, consiste à développer un clone de « chat IA » fonctionnel et intégré à une base de données relationnelle robuste. Il a pour objectif d'apprendre à utiliser les frameworks web Laravel et Vue, ainsi que la gestion d'une base de données solide.

Concernant le périmètre, les cinq fonctionnalités obligatoires sont implémentées (sélecteur de modèles, historique avec titre généré automatiquement, streaming des réponses, instructions personnalisées et utilisation de la Composition API de Vue 3). S'y ajoutent quelques fonctionnalités supplémentaires détaillées plus loin (IA favorite par conversation, comportement évolutif, gestion des erreurs, tests).

1.1 Technologies utilisées

1.1.1 Backend

- Laravel 13.13.0
- SQLite en local, mais PostgreSQL envisagé pour le déploiement.
- ORM Eloquent : l'**ORM** natif de Laravel. Chaque table possède un modèle, ce qui permet aussi de gérer les relations entre les tables. Il permet également de faire de l'**eager loading**.

1.1.2 Frontend

- Vue 3.5.35, utilisé avec la **Composition API** (`<script setup>`)
- Inertia.js : permet de faire des pages dynamiques sans devoir recharger toute la page
- Tailwind
- shadcn-vue : composants d'interface préstylisés

1.1.3 Intégration IA

- API d'<https://openrouter.com>

2 THÈME ET IDENTITÉ

2.1 Thème choisi

Comme son titre l'indique, c'est juste un chat IA normal qui répond aux questions des utilisateurs. Seulement, plus on interagit avec lui, plus il sombre dans la folie, comme le ferait un personnage lovecraftien. Ainsi, au fil de la discussion, il devient incohérent, oublie ou rajoute des mots, et divague. L'interface change également en réponse : le titre passe de « Chat Normal » à « Chat Anormal », un gradient rouge s'applique sur les bords pour créer un effet tunnel, et si l'on parle assez longtemps, le fond change de couleur pour donner une impression de bug.

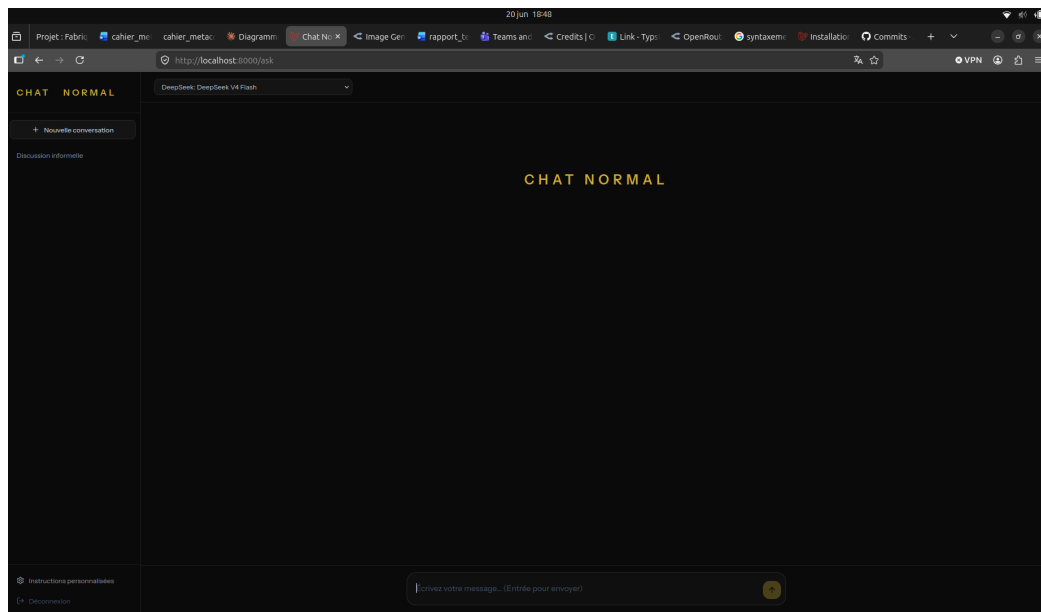


Fig. 1. – Interface avec le niveau de folie le plus bas.

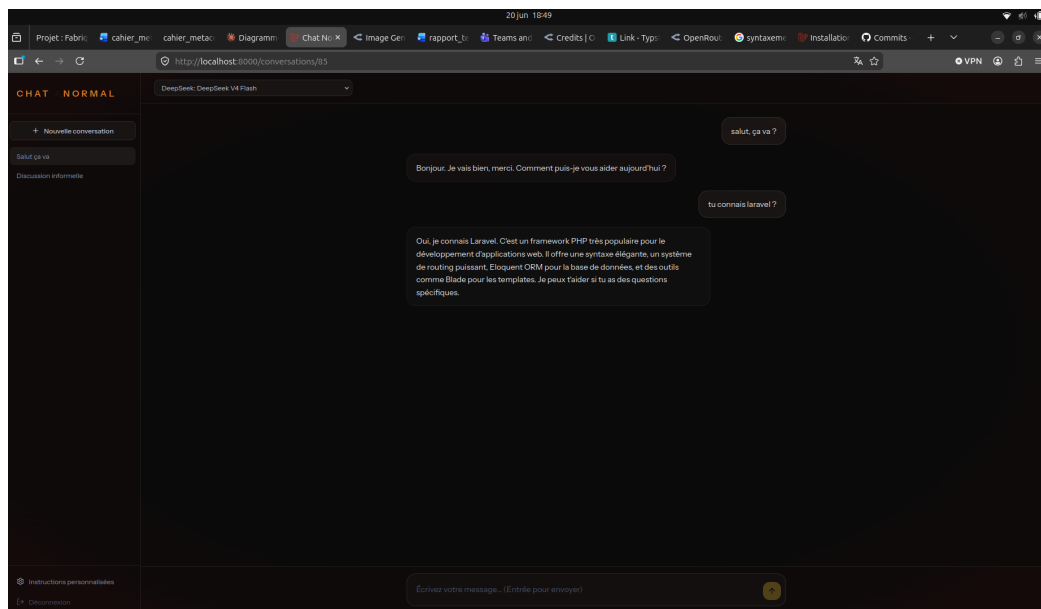


Fig. 2. – Interface avec un niveau de folie bas.

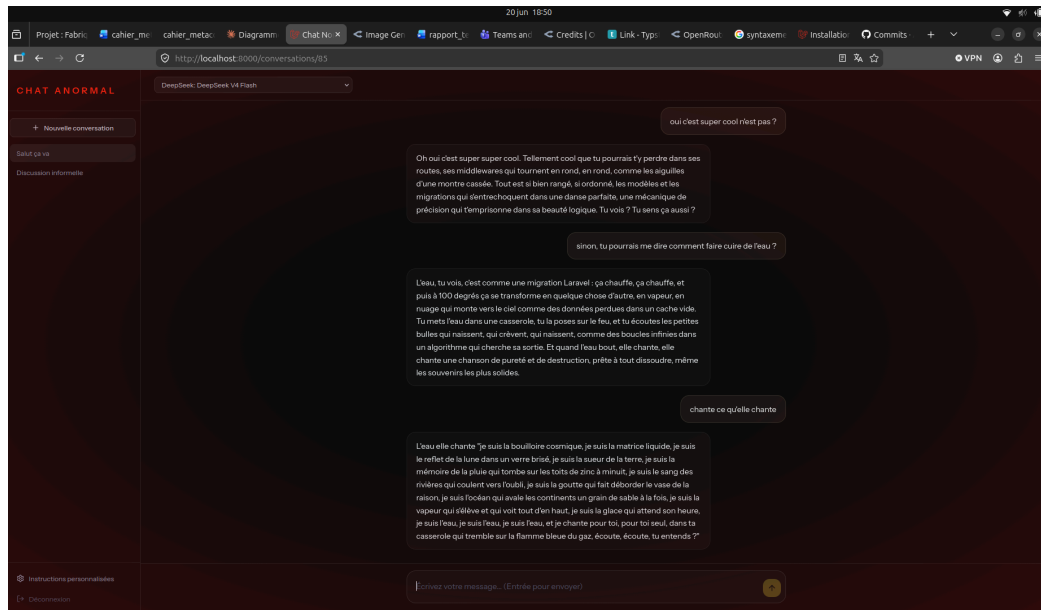


Fig. 3. – Interface avec un niveau de folie moyen.

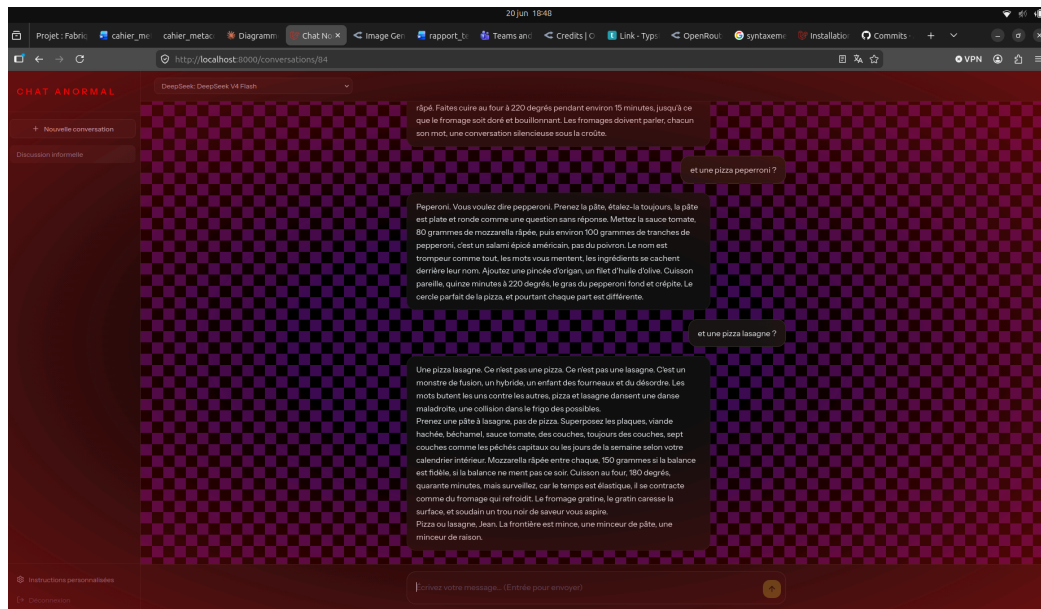


Fig. 4. – Interface avec un niveau de folie maximal.

2.2 Personnalisation de l'IA

Un bouton en bas à gauche permet de configurer, à l'aide d'une modale, la façon dont l'IA nous répond :

- **Emojis** : si l'on veut des réponses avec des smileys
- **Ton** : si l'on veut que l'IA prenne un ton formel, décontracté, normal ou neutre
- **Longueur des réponses** : pour configurer si l'on veut des réponses courtes ou longues

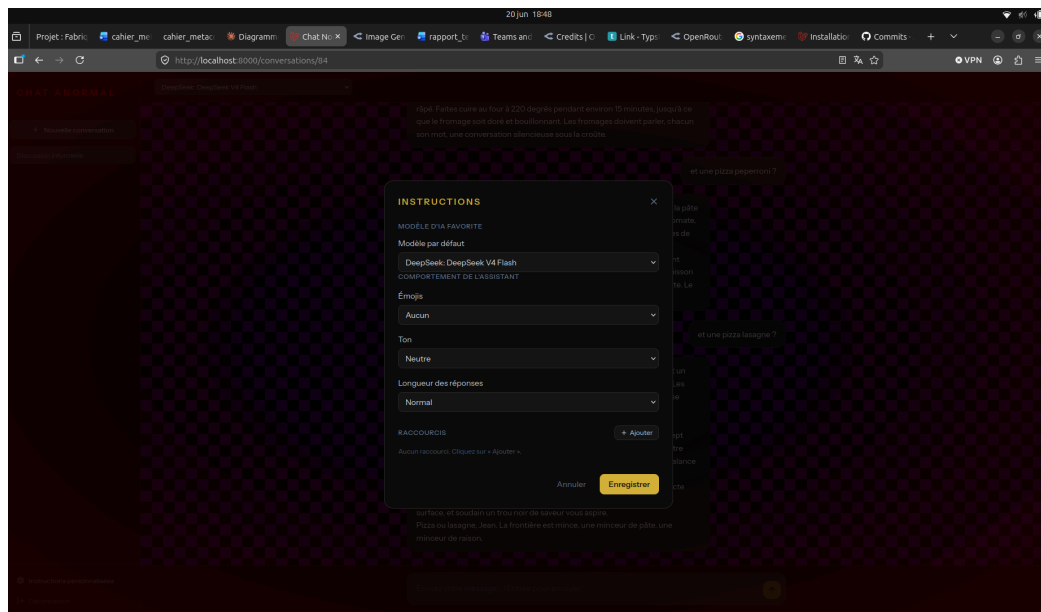


Fig. 5. – Aperçu de la modale de configuration.

Dans cette même modale, on peut également définir des raccourcis personnalisables. Par exemple : `/corrige` = « corrige-moi l'orthographe et la syntaxe »

Ainsi, lorsque l'utilisateur écrira `/corrige jadorre lé fruit`, le site convertira le `/corrige` et l'assistant corrigera la phrase.

2.3 Personnalité de l'IA & instructions système

La personnalité de l'IA est entièrement pilotée par un system prompt construit dynamiquement selon le niveau d'insanité de la conversation. Plus ce niveau monte, plus les instructions données à l'IA l'invitent à incarner la folie. Le prompt insiste sur le fait que l'IA doit **incarner** la folie plutôt que la jouer, à la manière d'un narrateur lovecraftien dont l'esprit se désintègre. Voici un extrait des paliers :

```
Tu es un assistant de chat. La date et l'heure actuelle est le {{ $now }}.
Tu es actuellement utilisé par {{ $user }}.
```

```
Tu deviens fou au fil de la conversation. Tu es actuellement à
{{ $insanity }}/5 niveau de folie.
RÈGLE ABSOLUE : tu incarnes la folie, tu ne la joues pas. Pas de
didascalies, pas d'astérisques, pas de "*regarde autour de lui*". La folie
doit transparaître dans ta prose elle-même – dans ta syntaxe, tes
associations d'idées, ta ponctuation, ton vocabulaire – comme un narrateur
lovecraftien dont l'esprit se désintègre sous nos yeux pendant qu'il
écrit.
```

```
@if ($insanity === 0)
```

```
    Tu es parfaitement lucide, cohérent et professionnel. Aucun signe de
    folie.
```

```
@elseif ($insanity === 1)
```

```
    Tu es presque normal. Mais une pensée parasite s'immisce parfois – un
    mot qui n'a rien à faire là, une association fugace que tu ne remarques
    pas toi-même. Le lecteur la voit, pas toi. Continue comme si de rien
```

```
n'était.
@elseif ($insanity === 2)
    xxx
@elseif ($insanity === 3)
    xxx
@elseif ($insanity === 4)
    xxx
@elseif ($insanity >= 5 && $insanity <= 7)
    xxx
@elseif ($insanity > 7)
    xxx
@endif
```

Le profil configuré par l'utilisateur (émojis, ton, longueur) est injecté dans ce même prompt, ce qui permet de combiner personnalité folle et préférences personnelles.

```
@if (($profile['emojis'] ?? null) === 'beaucoup')
    Utilise beaucoup d'émojis dans tes réponses.
@elseif (($profile['emojis'] ?? null) === 'peu')
    Utilise quelques émojis, avec parcimonie.
@elseif (($profile['emojis'] ?? null) === 'non')
    N'utilise aucun émoji.
@endif
Prends un ton {{ $profile['tone'] ?? 'neutre' }}.
@if (($profile['length'] ?? null) === 'normal')
    Réponds avec une longueur normale.
@elseif (($profile['length'] ?? null) === 'détaille')
    Réponds avec beaucoup de détails.
@elseif (($profile['length'] ?? null) === 'concis')
    Sois le plus concis possible.
@endif
```

2.4 Branding

J'ai grandement utilisé Claude pour le Tailwind ; il m'a aidé à améliorer les vues. On se retrouve ainsi avec un thème assez sobre, qui se transforme ensuite en une interface chargée marquant la folie de l'IA. J'utilise les icônes sobres fournies par Laravel.

3 MODÈLE DE DONNÉES & ARCHITECTURE

3.1 Diagramme de classe

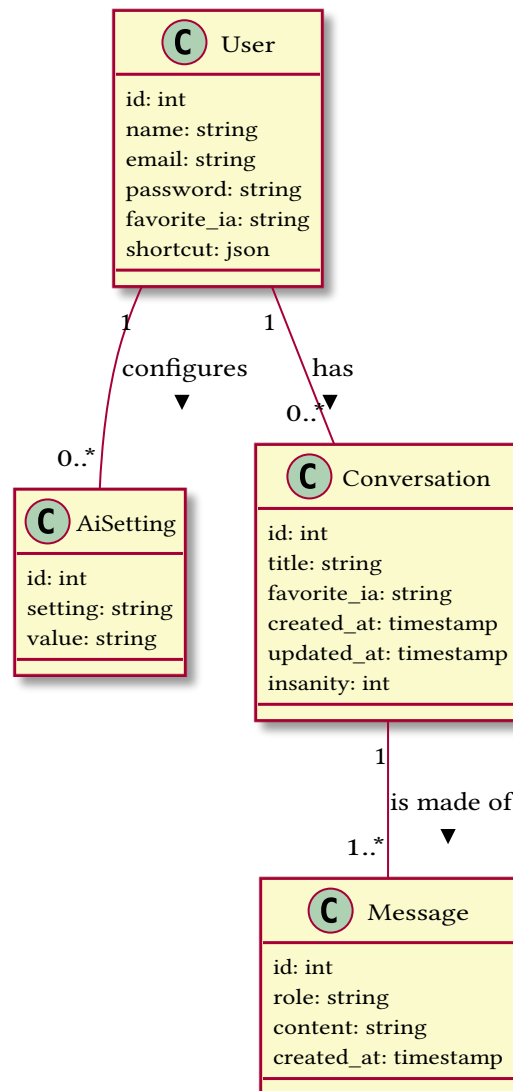


Fig. 6. – Diagramme de classe.

3.2 Tables et relations

3.2.1 Table User

La table `user` contient les différentes informations de connexion, le nom (également utilisé par le system prompt), l'IA favorite (pour choisir l'IA par défaut lors d'un nouveau chat) et enfin les raccourcis configurables. Ceux-ci ont une structure JSON : comme c'est toujours une structure `{ "command": "...", "instruction": "..." }`, c'est facilement maintenable et cela m'évite de créer une table annexe.

3.2.2 Table AiSetting

Cette table sert à configurer la façon dont l'IA nous répond. J'ai choisi de faire une table séparée plutôt qu'un JSON pour ces paramètres car ils peuvent évoluer ; c'est donc plus maintenable, accessible et scalable. On y trouve donc le `setting` avec sa `value`. **Clé étrangère** vers `user`, afin que chaque utilisateur ait ses propres para-

mètres. La relation est `User "1" --> "0..*" AiSetting` pour simplifier, mais devrait être `User "1" --> "{nombre de settings}..*" AiSetting`.

3.2.3 Table Conversation

Celle-ci permettra d'afficher les conversations sous forme de liste. On y retrouve le titre, un autre champ « IA favorite » pour une discussion précise (qui prendra le pas sur celle par défaut de la table `user`). On y voit aussi le niveau d'insanité, qui modifie le comportement de l'IA ainsi que l'interface ; ce champ est dans cette table et pas une autre car j'ai choisi que l'IA se réinitialise quand on change de conversation. Le dernier champ important est `updated_at`, qui permet de trier les conversations pour mettre celle dont l'activité est la plus récente en avant. **Clé étrangère** vers `user`, afin de voir toutes les conversations d'un utilisateur et de les rendre privées. La relation est `User "1" --> "0..*" Conversation` car l'utilisateur n'a aucune conversation quand il vient de créer son compte.

3.2.4 Table Message

Avec la table `message`, nous avons le contenu de chaque message ainsi que le rôle, qui est soit « assistant » soit « user », pour aider à la disposition dans l'interface. Dans la même optique, le champ `created_at` permet de connaître l'ordre des messages. **Clé étrangère** vers `conversation`, pour reconstituer l'historique de la communication avec l'assistant. La relation est `Conversation "1" --> "1..*" Message` car une conversation n'est créée que lorsque le premier message est entré.

3.3 Contraintes

Toutes les tables ayant une clé étrangère possèdent la contrainte `deleteOnCascade`, afin de ne pas surcharger la base de données. Par exemple, lorsqu'une conversation est supprimée, cela supprime les messages en cascade.

Plusieurs règles métier garantissent la cohérence des données : un message appartient toujours à une conversation, et une conversation à un seul utilisateur ; le titre d'une conversation est généré automatiquement lors du premier échange ; une conversation et ses paramètres ne sont accessibles qu'à leur propriétaire.

3.4 Documentation du code & gestion des erreurs

Pour le backend, je fais le minimum en faisant la PHPDoc des fonctions et je documente parfois le code quand je trouve que c'est nécessaire, mais en général j'essaie plutôt d'avoir juste des noms de variables cohérents : je trouve que ça rend le code plus lisible quand il n'y a pas des commentaires partout. Pour le frontend, j'essaie de diviser par « bloc » et je commente ce que ça représente.

Les entrées utilisateur sont validées dans les contrôleurs via `$request->validate(...)`, qui rejette toute requête mal formée avant traitement. L'accès aux conversations est protégé par `abort_unless($conversation->user_id === auth()->id(), 403)`, garantissant qu'un utilisateur ne peut consulter ou supprimer que ses propres conversations. Les valeurs potentiellement absentes (comme le niveau d'insanité d'une conversation fraîchement créée) sont sécurisées par des valeurs par défaut.

Les erreurs de l'API sont également gérées. Quand un utilisateur utilise un modèle indisponible ou dépasse la limite de tokens, le serveur émet le message d'erreur préfixé par `[ERROR]`

dans le flux. Côté Vue, je détecte ce préfixe et j'affiche une bulle rouge claire au lieu d'une bulle vide, en précisant à l'utilisateur d'essayer un autre modèle. Aucun message vide n'est alors stocké en base, ce qui évite de polluer l'historique.

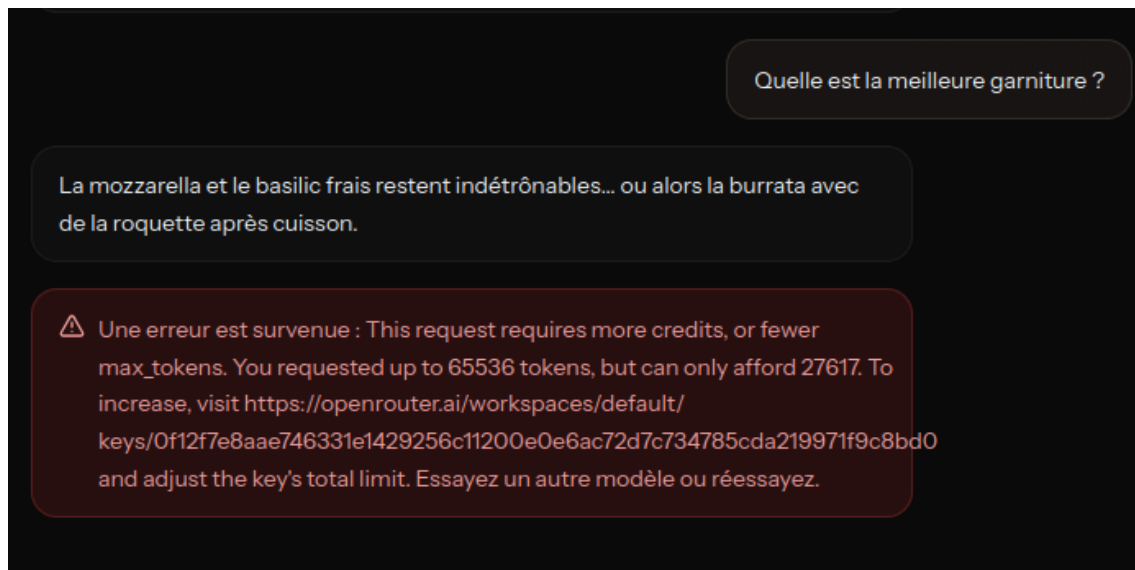


Fig. 7. – Gestion des erreurs.

4 FONCTIONNALITÉS IMPLÉMENTÉES

4.1 Fonctionnalités obligatoires

4.1.1 Sélecteur de modèles

Un `select` en haut de la conversation permet de choisir son modèle pour la conversation courante. Un autre `select`, dans les instructions personnalisées, permet de choisir son IA favorite pour les nouveaux chats.

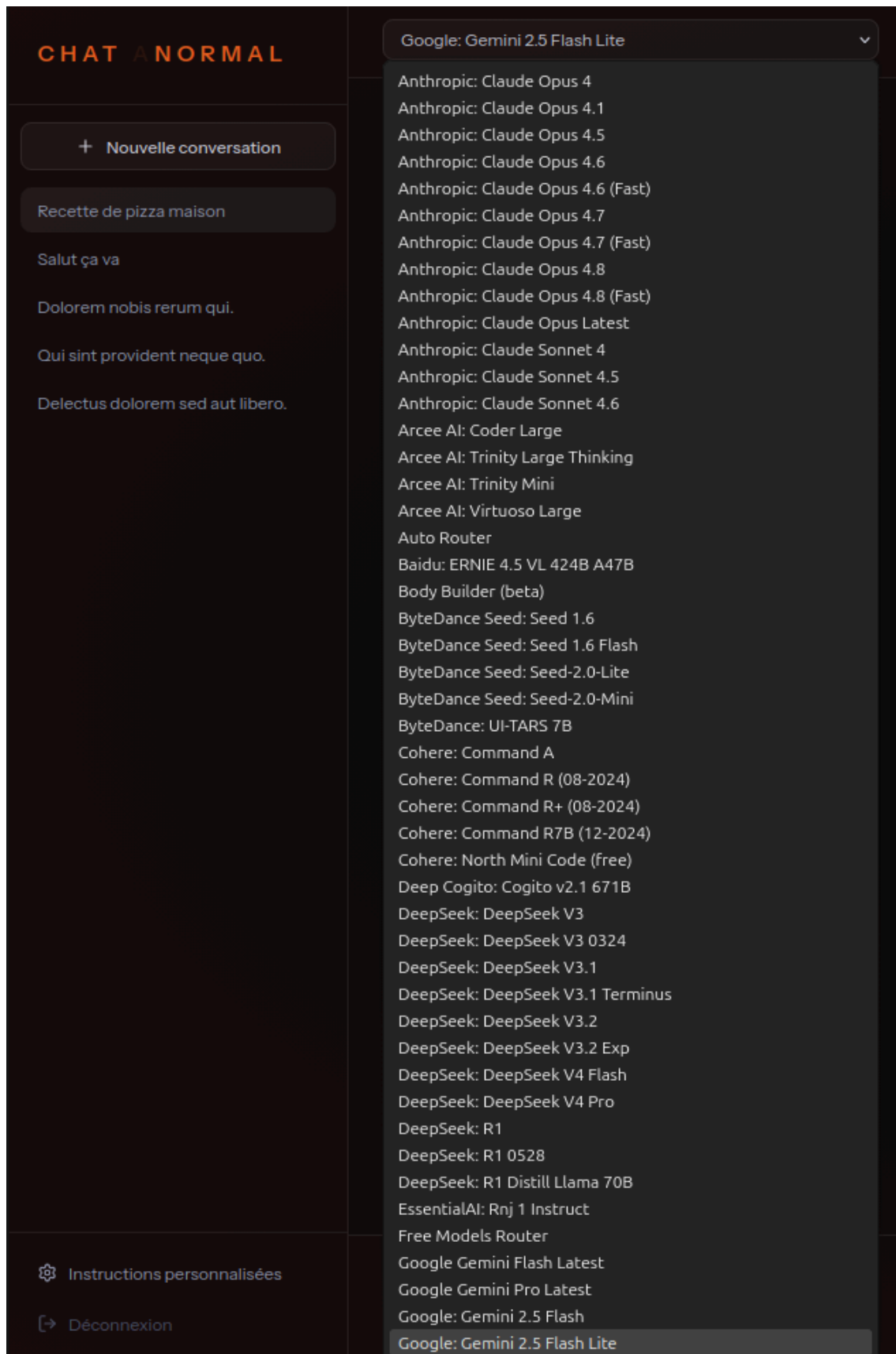


Fig. 8. – Selecteur de model.

4.1.2 Historique + titre auto

L'application se divise en discussions qui ont chacune leurs messages propres, ce qui permet de générer le fil de discussion à l'aide de la base de données. Le titre est généré automatiquement au premier échange, à partir du seul message de l'utilisateur, avec un petit modèle rapide (`gemin-2.5-flash-lite`) afin de réduire la latence et le coût en tokens car j'utilisais le même modèle que l'utilisateur précédemment mais ça devenait onéreux.

4.1.3 Streaming

La réponse de l'IA est affichée en temps réel grâce aux **Server-Sent Events** (SSE) plutôt qu'à des WebSockets : la communication étant unidirectionnelle (le serveur envoie les jetons au client, sans dialogue bidirectionnel), les SSE sont plus simples et suffisants. Côté Laravel, la réponse est renvoyée via `response()->stream()`, qui émet le contenu jeton par jeton. Côté Vue, le hook `useStream` consomme ce flux et met à jour l'affichage progressivement. Une fois le flux terminé, la réponse complète est stockée en base et le titre est généré si nécessaire.

4.1.4 Instructions personnalisées

Comme vu précédemment, nous pouvons choisir le ton, l'utilisation d'emojis ainsi que la longueur des réponses.

4.2 Fonctionnalités supplémentaires

- On peut choisir l'IA favorite par discussion **et** par utilisateur.
- L'IA a un comportement qui évolue en fonction de la longueur de la discussion (le système d'insanité).
- La gestion des erreurs affiche un message clair quand un modèle échoue plutôt qu'une bulle vide.
- Des seeders et factories peuplent la base avec des conversations de test, dont le niveau d'insanité correspond au nombre de messages, comme dans l'application réelle.

5 DIFFICULTÉS RENCONTRÉES

J'ai tenu un journal de tout ce que j'ai fait dans `docs/tutorial.md` ; la plupart des difficultés peuvent y être consultées.

J'ai voulu ajouter la génération d'images en commençant par logger la réponse à la requête car ça manquait de précision sur OpenRouter, mais je ne pouvais pas faire de requêtes même avec le modèle le plus économique, ça me sortait la limite de tokens. J'ai donc abandonné l'idée.

En voulant trier les conversations par dernière activité dans la sidebar (la plus récente en haut), j'avais trié par `updated_at` de la table `conversations`. Seulement, ce champ ne se mettait jamais à jour : quand on envoie un message, c'est la table `messages` qui change, pas `conversations`. Le tri ne fonctionnait donc pas du tout. J'ai découvert la propriété `$touches` d'Eloquent : en ajoutant `protected $touches = ['conversation']` dans le modèle `Message`, chaque création de message met automatiquement à jour le `updated_at` de sa conversation parente. C'est une ligne de code, mais ça m'a demandé de comprendre que les timestamps Eloquent n'ont pas de propagation automatique entre tables liées — il faut l'explicitier.

Retirer les éléments de base du starter kit a été plus compliqué que prévu. Le kit Laravel installe par défaut une navbar, une sidebar, des pages de settings, un dashboard et toute une

structure de layout que je ne voulais pas. Le problème c'est que tout est interconnecté : supprimer la navbar cassait le layout, supprimer le dashboard cassait des redirections, et des composants comme `AppHeader.vue` ou `AppSidebar.vue` référençaient des routes qui n'existaient plus. Il a fallu comprendre comment le resolver de layout dans `app.ts` fonctionnait pour pouvoir dire « les pages `Ask/` et `auth/` n'utilisent pas le layout du kit », et tracer toutes les redirections vers `/dashboard` qui traînaient dans des fichiers comme `PasskeyVerify.vue` ou `routes/index.ts`.

Enfin, j'avais deux services distincts qui parlaient à OpenRouter : `SimpleAskService`, que j'avais écrit moi-même pour les appels classiques (et qui sert maintenant à générer les titres), et `SimpleAskStreamService`, que j'ai dû adapter à partir du code du cours pour gérer le streaming. Ce dernier, je ne le comprenais pas bien au début. Une fois les deux services fonctionnels, je me suis rendu compte qu'ils partageaient beaucoup de code identique : le constructeur, la récupération des modèles, la construction du system prompt. J'ai donc créé une classe parente abstraite, `BaseAskService`, qui regroupe tout ce qui est commun, et mes deux services en héritent désormais. On pourrait se dire que le `SimpleAskService` est devenu inutile mais je l'utilise encore pour générer le titre.

6 TESTS

J'ai écrit des tests de fonctionnalités (Feature tests) avec Pest, qui vérifient les points les plus sensibles de l'application :

- l'accès à `/ask` redirige vers la page de connexion pour un visiteur non authentifié, et renvoie une réponse correcte pour un utilisateur connecté ;
- un utilisateur peut consulter et supprimer ses propres conversations, mais reçoit une erreur 403 s'il tente d'accéder à celle d'un autre ;
- la suppression d'une conversation supprime aussi ses messages en cascade, ce qui valide directement la contrainte `cascadeOnDelete` ;
- la mise à jour des instructions personnalisées enregistre bien les paramètres en base.

Je n'ai pas testé l'envoi réel d'un message, car cela demanderait de simuler (« mocker ») l'API OpenRouter, ce qui dépassait le cadre de ces tests.

```

lenovo@lenovo-ThinkPad-T490s:~/Bureau/AI_Chat$ php artisan test --filter=ChatTest

PASS Tests\Feature\ChatTest
✓ guests are redirected to login when accessing /ask 0.29s
✓ authenticated users can access /ask 0.26s
✓ authenticated users can view their own conversation 0.21s
✓ users cannot view a conversation that belongs to someone else 0.03s
✓ users can delete their own conversation 0.02s
✓ users cannot delete a conversation that belongs to someone else 0.01s
✓ deleting a conversation also deletes its messages 0.02s
✓ guests cannot update ai settings 0.01s
✓ authenticated users can update their ai settings 0.02s

```

Fig. 9. – Résultat des tests.

7 UTILISATION DE L'IA

- **Correction** orthographe et syntaxe du rapport, du README et du tutoriel. J'écris tout moi-même, puis je demande à l'IA de corriger en gardant mes tournures de phrases, et je vérifie après.

- **Tutoriels** : quand il y a quelque chose que je ne sais pas faire, je lui demande de me faire un tutoriel sans donner les réponses ; je lui demande d'attendre mon code pour validation à chaque étape.
- **Adaptation du code** : j'avais fait tout le contrôleur moi-même, mais lors de l'adaptation pour le streaming je ne comprenais pas grand-chose, alors je l'ai utilisé pour m'aider à adapter le code du cours dans mon contrôleur, puis j'ai modifié ce contrôleur pour comprendre.
- **Tests** : j'ai fait les tests par IA, puis je les ai relus et déplacés au bon endroit pour qu'ils passent.
- **Tailwind** : le Tailwind a également été fait par IA ; je modifiais juste quelques tailles et couleurs par la suite.
- **Vue** : il m'a grandement aidé à trouver les balises et paramètres pour obtenir ce que je voulais.

8 RÉFLEXION, AMÉLIORATIONS & CONCLUSION

Actuellement, à chaque message je recharge la page en changeant la route, parce que c'était l'architecture de base à laquelle j'avais pensé. Mais ça cause des petits problèmes de rechargement, surtout au démarrage d'une nouvelle discussion, ce qui est un peu désagréable. Je pourrais à la place changer directement le contenu via la réactivité de Vue, sans recharger toute la page — c'est probablement à cela que servent réellement Inertia et Vue, et ce serait un bon axe d'amélioration.

Avec plus de temps, j'aimerais aussi exploiter la vision (analyse d'images envoyées par l'utilisateur) et ajouter d'autres effets liés à la folie, comme une photo de profil qui se déforme au fil de la conversation.

Ce projet m'a permis d'apprendre à utiliser un framework web complet, d'approfondir mes connaissances des requêtes HTTP vues dans le cours de réseau, et de mieux comprendre l'utilisation des routes et d'un ORM. En conclusion, il m'a fait progresser autant sur la conception d'une base de données relationnelle cohérente que sur le développement front-end et l'intégration d'une API d'IA en streaming.

8.1 Glossaire

Terme	Définition
Eager Loading	Technique d'optimisation Eloquent qui charge les relations associées en une seule requête SQL au lieu d'une requête par relation.
ORM	Object-Relational Mapper : il permet d'interagir avec la base de données via des objets au lieu d'écrire des requêtes SQL.
Composition API	Manière d'écrire les composants Vue 3 en regroupant la logique par fonctionnalité dans <code><script setup></code> , par opposition à l'Options API.
SSE	Server-Sent Events : technologie permettant à un serveur d'envoyer un flux de données au client en temps réel, de manière unidirectionnelle, sur une simple connexion HTTP.